

Q Méthode : Comprendre un message d'erreur (Traceback)

Lorsqu'une exception survient, Python interrompt l'exécution et affiche une trace d'appels (*traceback*) qui se lit **de bas en haut** :

- ❶ **Lire la dernière ligne en premier** : elle donne le **type de l'exception** et un message explicatif précis (ex : `IndexError: list index out of range`).
- ❷ **Remonter à la ligne incriminée** : repérer la ligne File "`programme.py`", line `X`, in ... correspondant à *votre* fichier (et non aux fichiers internes de la bibliothèque standard).
- ❸ **⚠ Cas particulier du SyntaxError** : l'erreur réelle se situe très souvent à la ligne **précédente** (ex : parenthèse ou guillemet non fermé à la ligne `X - 1`, l'interpréteur ne détecte l'anomalie qu'à la ligne `X`).

🔧 1. Erreurs de syntaxe (SyntaxError)

Une erreur de syntaxe empêche l'exécution du script avant même son démarrage (phase d'analyse lexicale et syntaxique).

- **Oubli des deux-points** : à la fin d'un en-tête d'instruction conditionnelle, de boucle ou de fonction :

❌ Code erroné :

```
1 if x > 0
2     print("positif")
3 def carre(n)
4     return n * n
```

✅ Code corrigé :

```
1 if x > 0:
2     print("positif")
3 def carre(n):
4     return n * n
```

- **Confusion entre affectation = et comparaison ==** :

❌ Code erroné :

```
1 if a = 3:
2     print("égal à 3")
```

✅ Code corrigé :

```
1 if a == 3:
2     print("égal à 3")
```

- **Parenthèse, crochet ou guillemet non fermé** :

❌ Code erroné :

```
1 val = (2 * x + 1
2 print(val) # SyntaxError ici !
```

✅ Code corrigé :

```
1 val = (2 * x + 1)
2 print(val)
```

- **Mots-clés erronés ou syntaxe empruntée à d'autres langages** :

Syntaxe erronée	Syntaxe Python	Explication / Contexte
else if / elsif	elif	Branche alternative conditionnelle.
true / false	True / False	Les booléens prennent obligatoirement une majuscule.
null	None	Valeur / objet représentant l'absence de valeur.
&& / / !	and / or / not	Opérateurs logiques booléens en toutes lettres.
function / let / var	def / (direct)	Définition de fonction ou affectation de variable.

- **Noms de variables invalides** : un identificateur ne peut ni commencer par un chiffre (ex : `1ere_valeur`), ni contenir de tiret (ex : `total-points` confondu avec une soustraction), ni être un mot réservé Python (`for`, `in`, `def`, `class`, `return`, `pass`, `while`, `import`, ...).

🛠 2. Erreurs d'indentation (IndentationError)

En Python, l'indentation structure les blocs de code. Toute incohérence bloque l'exécution.

- **expected an indented block** : un bloc vide suit un deux-points `:`. Si le bloc doit rester vide (temporairement), utiliser le mot-clé **pass**.

❌ Code erroné :

```
1 def a_completer(x):
2     # Bloc vide -> Erreur !
3     def suivante(y):
4         return y + 1
```

✅ Code corrigé :

```
1 def a_completer(x):
2     pass # Corps temporaire
3     def suivante(y):
4         return y + 1
```

- **unindent does not match any outer indentation level** : décalage d'indentation incohérent ou mélange imperceptible d'espaces et de tabulations.

❗ Toujours configurer son éditeur de code pour insérer **4 espaces** lors de l'appui sur la touche



🛠 3. Erreurs de type (TypeError)

Survient lors de l'application d'un opérateur ou d'une fonction à un objet d'un type non approprié.

- **unsupported operand type(s)** : opération interdite entre types incompatibles :

❌ Code erroné :

```
1 s = "Total : " + 42
2 # Concaténation str + int
3 L = [1, 2, 3] - [2]
4 # Soustraction sur listes
```

✅ Code corrigé :

```
1 s = "Total : " + str(42)
2 # Conversion explicite
3 L.remove(2)
4 # Méthode appropriée
```

- **Flottant utilisé comme indice ou paramètre de range** :

❌ Code erroné :

```
1 milieu = len(L) / 2
2 val = L[milieu] # float !
```

✅ Code corrigé :

```
1 milieu = len(L) // 2
2 val = L[milieu] # int !
```

- **'NoneType' object is not ...** : utilisation du résultat d'une fonction sans **return** ou réassignation du résultat d'une méthode modifiant en place :

❌ Code erroné :

```
1 L = [3, 1, 2]
2 L = L.append(4)
3 # L vaut maintenant None !
4 print(len(L)) # TypeError !
```

✅ Code corrigé :

```
1 L = [3, 1, 2]
2 L.append(4)
3 # append modifie L en place
4 print(len(L)) # Affiche 4
```

⚠ Même piège avec `L.sort()` (modifie en place et renvoie **None**) à distinguer de `sorted(L)` (renvoie une nouvelle liste triée).

- **'...' object is not callable** :

➤ Oubli du signe de multiplication : écrire `2 * (x + 1)` et non `2(x + 1)`.

➤ Utilisation de parenthèses au lieu de crochets pour accéder à un élément : `tab[i]` et non `tab(i)`.

➤ Écrasement d'une fonction native par une variable locale :

```
1 list = [1, 2, 3] # Variable masquant la fonction intégrée list
2 autre = list("abc") # TypeError: 'list' object is not callable !
```

- **takes X positional arguments but Y were given** : mauvais nombre d'arguments passés lors de l'appel d'une fonction.

- **'tuple' / 'str' object does not support item assignment** : tentative de modification d'un élément d'un type **immuable** (ex : `texte[0] = 'A'` ou `mon_tuple[1] = 10`).

🛠 4. Erreurs d'indice (IndexError)

L'exception `IndexError: list index out of range` survient lorsqu'on tente d'accéder à un élément avec un indice $i \notin \llbracket -\text{len}(L); \text{len}(L) - 1 \rrbracket$.

- Boucle allant un cran trop loin :

❌ Code erroné :

```
1 # Indice len(L) inexistant !
2 for i in range(len(L) + 1):
3     print(L[i])
```

✅ Code corrigé :

```
1 # Indices de 0 à len(L)-1
2 for i in range(len(L)):
3     print(L[i])
```

- Accès au successeur `L[i+1]` : la borne supérieure de la boucle doit être réduite d'une unité :

❌ Code erroné :

```
1 # Pour i = len(L)-1, i+1 déborde !
2 for i in range(len(L)):
3     if L[i] == L[i+1]:
4         print("doublon")
```

✅ Code corrigé :

```
1 # Arrêt à len(L)-2
2 for i in range(len(L) - 1):
3     if L[i] == L[i+1]:
4         print("doublon")
```

- Affectation directe sur une liste vide / non dimensionnée :

❌ Code erroné :

```
1 carres = []
2 for i in range(10):
3     carres[i] = i**2 # IndexError !
```

✅ Code corrigé :

```
1 carres = []
2 for i in range(10):
3     carres.append(i**2)
```

❗ Pour pré-allouer une liste de taille n , écrire `carres = [0] * 10`, puis `carres[i] = i**2`.

- Remarque sur les tranches : les tranches `L[debut:fin]` ne lèvent **jamais** d'`IndexError`, même si `fin` dépasse la longueur de la liste (l'extraction s'arrête simplement au dernier élément).

🔧 5. Erreurs de nom et portée (NameError, UnboundLocalError)

Survient lorsqu'on utilise un identificateur non défini ou non accessible dans la portée courante.

- **NameError**: name `'...'` is not defined :

- **Faute de frappe ou casse non respectée** : Python est sensible à la casse (`total`, `Total` et `TOTAL` sont 3 variables distinctes).
- **Utilisation avant affectation** : variable manipulée avant sa ligne d'initialisation.
- **Oubli d'un import** : utiliser `sqrt(x)` sans `from math import sqrt` préalable.
- **Variable locale inaccessible hors de sa fonction** :

❌ Code erroné :

```
1 def moyenne(a, b):
2     res = (a + b) / 2
3     return res
4
5 moyenne(12, 16)
6 print(res) # NameError !
```

✅ Code corrigé :

```
1 def moyenne(a, b):
2     res = (a + b) / 2
3     return res
4
5 m = moyenne(12, 16)
6 print(m) # Récupérer le résultat
```

- **Variable définie dans un bloc conditionnel non exécuté** : si une variable n'est affectée que dans un `if`, elle n'existera pas si la condition est fausse (lui donner une valeur par défaut avant le test).

- **ModuleNotFoundError**: No module named `'...'` : nom de bibliothèque mal orthographié (ex : `import maths` au lieu de `math`).
- **UnboundLocalError**: local variable referenced before assignment : survient lorsqu'une fonction tente de modifier une variable sans préciser qu'elle est globale, ou lorsqu'elle lit une variable locale avant son affectation :

❌ Code erroné :

```
1 compteur = 0
2 def incrementer():
3     compteur = compteur + 1
4 # Python considère compteur local
5 # mais il n'est pas encore assigné
```

✅ Code corrigé :

```
1 def incrementer(compteur):
2     return compteur + 1
3
4 compteur = 0
5 compteur = incrementer(compteur)
```

🔧 6. Valeurs, arithmétique et assertions (ValueError, ZeroDivisionError)

L'argument est du bon type, mais sa valeur est invalide ou non acceptée pour l'opération demandée.

- **ZeroDivisionError**: division by zero : division `/`, quotient entier `//` ou modulo `%` avec un dénominateur nul (cas d'une liste vide lors du calcul d'une moyenne, ou d'un pivot nul).
- **ValueError**: invalid literal for `int()` : tentative de conversion d'une chaîne ne représentant pas un entier :

❌ Code erroné :

```
1 n = int("3.14") # ValueError !
2 m = int("dix")  # ValueError !
```

✅ Code corrigé :

```
1 n = int(float("3.14")) # Vaut 3
2 m = 10
```

- **ValueError**: math domain error : argument hors du domaine de validité mathématique d'une fonction (ex : `math.sqrt(-4)`, `math.log(0)`, `math.asin(2)`).
- **ValueError**: not enough / too many values to unpack : déballage d'un tuple ou d'une liste avec un nombre incompatible de variables à gauche de l'affectation :

❌ Code erroné :

```
1 x, y = (1, 2, 3) # Trop de valeurs
2 a, b, c = [10, 20] # Pas assez
```

✅ Code corrigé :

```
1 x, y, z = (1, 2, 3)
2 a, b = [10, 20]
```

- **AssertionError** : levé lorsqu'une instruction `assert <condition>` échoue (la précondition testée n'est pas vérifiée lors de l'appel).

7. Fichiers, dictionnaires et attributs (KeyError, AttributeError)

- **FileNotFoundError**: [Errno 2] No such file or directory : tentative d'ouverture avec `open("fichier.txt")` d'un fichier inexistant, mal nommé (ex : faute dans l'extension) ou situé dans un autre répertoire (problème de chemin relatif).

- **KeyError**: 'cle' : accès direct à une clé qui n'existe pas dans le dictionnaire :

✖ Code erroné :

```
1 d = {"a": 1, "b": 2}
2 val = d["c"] # KeyError !
```

✔ Code corrigé :

```
1 if "c" in d:
2     val = d["c"]
3 val = d.get("c", 0) # Avec défaut
```

- **AttributeError**: '...' object has no attribute '...' :

- Utilisation d'une méthode inexistante sur le type manipulé (ex : `L.push(5)` au lieu de `L.append(5)`, `s.append("a")` sur une chaîne, ou `tab.length` au lieu de `len(tab)`).
- Appel de méthode sur `None` consécutif à une fonction sans `return` : **AttributeError**: 'NoneType' object has no attribute 'split'.

8. Récursion et boucles infinies (RecursionError et terminaison)

- **RecursionError**: maximum recursion depth exceeded : trop grand nombre d'appels récursifs imbriqués (limite par défaut à 1000) :

- **Oubli du cas de base (condition d'arrêt)** : la fonction s'appelle indéfiniment.
- **Pas de récurrence incorrect** : l'argument ne converge pas vers le cas de base (ex : appel `f(n)` au lieu de `f(n - 1)`).

✖ Code erroné :

```
1 def fact(n):
2     # Oubli du cas de base !
3     return n * fact(n - 1)
```

✔ Code corrigé :

```
1 def fact(n):
2     if n == 0:
3         return 1
4     return n * fact(n - 1)
```

- **Boucle while infinie** : le programme s'exécute sans fin et bloque sans message d'erreur :

- Oubli de la mise à jour de la variable de contrôle dans le corps du `while` (ex : omission de `i += 1`).
- Condition d'arrêt stricte sur un flottant : `while x != 1.0` : avec un pas `x += 0.1` peut ne jamais s'arrêter en raison des erreurs d'arrondi de l'arithmétique flottante.

⚠ 9. Pièges logiques classiques (Aucune exception mais résultat faux !)

- Aliasing et duplication de listes (partage de références) :

❌ Code erroné :

```
1 a = [1, 2, 3]
2 b = a # Même objet en mémoire !
3 b.append(4)
4 # a vaut aussi [1, 2, 3, 4] !
```

✅ Code corrigé :

```
1 a = [1, 2, 3]
2 b = list(a) # ou b = a[:] / a.copy()
3 b.append(4)
4 # a reste [1, 2, 3]
```

- Création de matrices / grilles 2D avec * :

❌ Code erroné :

```
1 # 3 fois la MÊME ligne !
2 grille = [[0] * 3] * 3
3 grille[0][0] = 1
4 # Modifie toute la colonne 0 !
```

✅ Code corrigé :

```
1 # 3 lignes indépendantes
2 grille = [[0]*3 for _ in range(3)]
3 grille[0][0] = 1
4 # Seule la case (0,0) vaut 1
```

- Modification d'une liste pendant son parcours :

❌ Code erroné :

```
1 L = [1, 2, 2, 3]
2 for x in L:
3     if x == 2:
4         L.remove(x) # Saute un '2' !
```

✅ Code corrigé :

```
1 L = [1, 2, 2, 3]
2 # Filtrer par compréhension :
3 L = [x for x in L if x != 2]
```

- Oubli de `return` dans une fonction récursive :

❌ Code erroné :

```
1 def recherche(x, L, i=0):
2     if i >= len(L):
3         return False
4     if L[i] == x:
5         return True
6     # Oubli du return !
7     recherche(x, L, i + 1)
8     # Renvoie None si trouvé plus loin
```

✅ Code corrigé :

```
1 def recherche(x, L, i=0):
2     if i >= len(L):
3         return False
4     if L[i] == x:
5         return True
6     return recherche(x, L, i + 1)
```

- Test d'égalité exacte sur les flottants :

❌ Code erroné :

```
1 if 0.1 + 0.2 == 0.3:
2     print("Égal") # N'affiche RIEN !
3 # Car 0.1+0.2=0.30000000000000004
```

✅ Code corrigé :

```
1 import math
2 # Ou abs(a - b) < 1e-9 :
3 if math.isclose(0.1 + 0.2, 0.3):
4     print("Égal")
```